

Reviewing, Validating, and Maintaining Agent Output

Automated gates catch what they're built to catch, not everything.

What You'll Learn

- 1 Apply human-in-the-loop review even after automated gates pass
- 2 Recognize hallucinations, unsafe changes, and weak assumptions
- 3 Manage spec drift and keep documentation in sync with the codebase

Human-in-the-Loop Still Matters

Confident-looking output can still contain hallucinations, unsafe changes, or assumptions that were never actually verified, because a gate only checks what it was built to check, not everything that could go wrong.

Production systems warrant extra scrutiny beyond what any automated gate applies by default, since the cost of a mistake reaching production is far higher than the cost of one more careful read.

Managing Spec Drift

A spec, constitution, or instruction file that no longer matches the code stops being trustworthy documentation, and worse, it can actively mislead the next person, or agent, who reads it.

Keeping them in sync is what makes them a usable audit trail later, not just a snapshot of intent at one point in time that nobody bothered to update.

WHY IT MATTERS

The riskiest changes are the ones that look done.

Confident output and correct output are not the same thing: an agent can produce polished, well-structured code that is still wrong in a way that's hard to spot at a glance.

The gap between them is exactly what human review exists to catch, which is why this step doesn't get skipped even after every automated gate has already passed.

APPLYING THE CONCEPTS

GitHub Spec Kit

Keeping Specs and Constitution Current

`/speckit.analyze`

Re-run after implementation as an extra review pass, not just before, to catch drift between what was built and what was specified, in case the implementation quietly diverged from the plan along the way.

Closing the gap: When review surfaces a gap, update the constitution or the spec directly (via `/speckit.specify` or another `/speckit.clarify` pass) rather than letting the code silently diverge from them, so the documentation stays the thing you can actually trust.

`/speckit.converge`

Assesses the codebase against the feature's spec, plan, and tasks and appends any missed work as new tasks. Append-only: it never edits or deletes code directly, so it's a safe way to surface gaps without risking an unreviewed change.

NEWER IN TOOLKIT

APPLYING THE CONCEPTS

OpenSpec

Verify and Update

`/opsx:verify`

Validates implementation against the change's artifacts across Completeness, Correctness, and Coherence. Reports CRITICAL, WARNING, or SUGGESTION issues, so a team can triage by severity instead of treating every finding the same way.

`/opsx:update`

Keeps artifacts coherent when something changes: revise one and it reconciles the others, the drift problem handled directly instead of relying on someone remembering to update every related file by hand.

Module 6 at a Glance

	GitHub Spec Kit	OpenSpec
Post-implementation review	Re-run /speckit.analyze; update constitution/spec	/opsx:verify (Completeness/Correctness/Coherence)
Keeping artifacts in sync	Update constitution, spec, or clarify answers	/opsx:update
Shared goal	Confirm what got built still matches what was decided	Same

Key Takeaways

- Passing every gate is not the same as being correct, which is why human review still has a job to do afterward.
- Both frameworks exist to answer the same question after code lands: does it still match what was decided, and if not, why not?
- Keeping documentation current is what makes it worth trusting the next time someone, or some agent, reads it.